

15

Задача по композиции в языке программирования Java

15.1. ОПИСАНИЕ ЗАДАЧИ

Как известно, в языке программирования Java отсутствует понятие указателя. То есть все `non-primitive` типы (массивы, строки, объекты любого класса) реализованы как указатели, но выглядят так, будто они не являются указателями. Таким образом, к созданным строкам, массивам и всем объектам, созданным как экземпляры классов, можно обращаться только через указатели. Поля, переменные, параметры методов и созданные анонимные величины содержат (в выделенной для них памяти) либо примитивное значение, либо адрес структуры, на которую они ссылаются.

Таким образом, Java решает острые проблемы, связанные с указателями. Проблема «висящих» указателей — одна из наиболее распространённых проблем в языках программирования, требующая чрезмерно много времени при отладке. Проблема «мусора» (объекта, на который нет ссылок), хотя и не является часто встречающейся, но если происходит переполнение памяти в самый неподходящий момент, последствия будут плачевными. Эти проблемы не возникают в Java, поскольку программист явно не работает с указателями. Управление памятью (сборка мусора) — это задача среды выполнения программы, и программист не вмешивается в этот процесс.

Однако Java, элегантно решая вышеупомянутые проблемы, затрудняет решение одной проблемы этапа проектирования — проблемы агрегации. В процессе проектирования так или иначе моделируются реальные сущности или процессы. Каждая реальная сущность имеет соответствующие свойства и естественно определенные операции, которые применяются к сущностям данного типа. Если какое-либо свойство сущности является сложным, то есть само по себе представляет собой сущность, в языках программирования такое моделирование (и заданные отношения между этими объектами) называется агрегацией. В тексте слово «объект» используется в значении некоторой модели сущности реального мира. Таким образом, объект, который агрегирует, называется целым, а объект-свойство — частью. Например, документ агрегирует своё содержание, компьютер агрегирует устройство — процессор, автомобиль агрегирует двигатель и т. д.

Хотя объект, включённый в агрегацию, понимается как часть объекта, некоторые из таких объектов-частей могут физически не входить в состав включающего объекта. Например, автор книги находится вне книги, человек имеет банковский счёт, не включая его физически в себя, акционер исключительно владеет акциями, но они физически не входят в него, свидетельство о регистрации автомобиля может находиться вне автомобиля и т. д. Эти отношения, несомненно, являются отношениями агрегации, но по своей природе скорее концептуальными, чем физическими. Включающий объект по-прежнему означает целое, но его часть содержится в целом опосредованно.

В языках с указателями [1] отношение целого и его косвенной части (свойства) реализуется с помощью указателей; объект-часть этого отношения называется указателем-частью или связью-частью. Свойства, физически включённые в целое, реализуются путём прямого выделения (расширения) памяти для части целого; объект-часть, включённый в это отношение, называется «значением части» или «расширенной частью». Таким образом, в этих языках определяется физическая граница всего объекта. Всё, что находится внутри объекта (в пределах его границ), что физически входит в него — включено (развернуто) в нём как часть его значений, а всё, с чем объект имеет агрегацию, но что физически не входит в него, то есть не содержится в пределах (границ) объекта, — включено в него в виде указателей на соответствующие значения. В [3] типы объектов, значения которых развернуты внутри другого объекта, называются расширенными типами.

В языке объектно-ориентированного проектирования UML [2] на диаграмме классов для моделирования таких отношений используются два отношения ассоциации. Отношение агрегации используется для указания ссылки (не входящей в границы целого) на часть. Отношение композиции используется для обозначения связи между целым и его расширенной частью (см. рис. 15.1) [2].

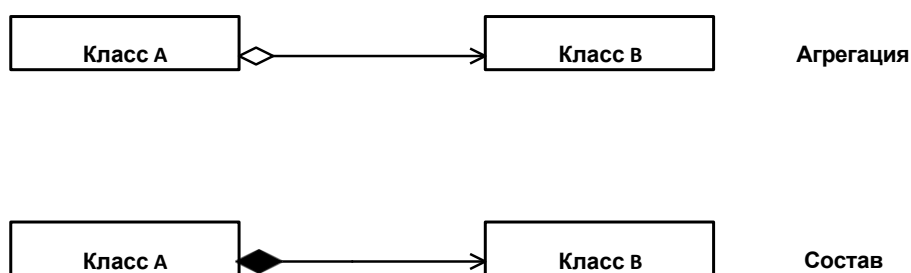


Рис. 15.1. Агрегация по ссылке (вверху) и по значению (внизу)

Источник: [2]

Фактически вместо указателя можно использовать ключ записи этого объекта в базе данных объектов данного класса. Это также агрегация, но без физического включения.

Понятие физических границ объекта необходимо для определения семантики выполнения некоторых базовых операций над объектом. К таким операциям относятся создание, присваивание, копирование (клонирование), сравнение и удаление. Например, при копировании объекта для частей-значений необходимо скопировать значения в эти объекты-части, а для частей-указателей — скопировать указатели на объекты-части, то есть скопировать адреса частей. Кроме того, физическая граница нужна для определения того, является ли часть объекта общей для нескольких объектов или же она является исключительно свойством своего владельца — объекта-целого. Существует множество примеров, когда части распределены между несколькими субъектами — владельцами: например, другая книга может указывать на этого автора, то есть две книги имеют одного и того же автора, поэтому обе книги ссылаются на одного и того же автора. Банковская книга счетов может указывать на счёт конкретного клиента, база данных в ГИБДД — на данное свидетельство о регистрации и т. д.

Если значение объекта изменяется, это изменение отражается во всех объектах, которые используют данный объект в качестве своей части и ссылаются на него через указатель. В реальной жизни так и должно быть. Действительно, если мы изменим личные данные автора книги, то изменятся данные во всех книгах этого автора. Аналогично, если изменить остаток или состояние на банковском счёте, это отразится как в бухгалтерских книгах клиента, так и в бухгалтерских книгах банка. Если данные в свидетельстве о регистрации автомобиля корректируются на основании данных ГИБДД (например, после перекраски), это изменение отражается в свидетельстве о регистрации автомобиля.

Концептуально вышеупомянутые операции (создание, присвоение и т. д.) над объектами основаны на двух проверках: проверке идентичности и проверке эквивалентности. Проверка идентичности — это проверка того, являются ли объекты одним и тем же объектом в реальном мире. Объекты одного и того же класса всегда отличаются друг от друга как сущности. Проверка эквивалентности — это проверка того, имеют ли объекты одинаковое состояние [3].

Исходя из этого, в языках с указателями вышеупомянутые операции над объектами естественным образом определяются стандартными операциями в пределах физических границ данной сущности. При работе с частями-указателями необходимо помнить, что объект-целое и объект-часть существуют отдельно друг от друга; экземпляры соответствующих классов можно создавать и уничтожать независимо друг от друга, а части-значения «рождаются» вместе с целым, «живут» в нём — это единственный объект, к которому у них есть доступ, — и «умирают» также вместе с целым.

В этих языках при создании объекта он создаётся в пределах своих физических границ. Для частей-значений выделяется место для объекта соответствующего класса, а для частей-указателей выделяется место только для указателя, независимо от того, к какому классу относится данная часть. При сравнении объекты сравниваются также в пределах своих физических границ: части-значения сравниваются как сравнение объектов, а для частей-указателей сравниваются адреса (значения указателей). Операция присваивания реализуется также в пределах физических границ: части-указатели копируются как значение указателя, то есть копируется адрес, а для частей-значений копируется объект. Операция копирования реализуется также точно так же, как операция присваивания.

Для языков с указателями, наряду с вышеупомянутым преимуществом в виде определения границ сущностей и естественного подхода к проектированию агрегации, существуют проблемы, которые необходимо решать. Помимо проблем с «висящими» ссылками и «мусором», имеются следующие моменты: чтобы избежать структурной зависимости через ссылки, важно соблюдать единый порядок создания и уничтожения объектов, ссылки на которые могут содержаться в разных местах. Необходимо, чтобы этим занимался один и тот же субъект. Кроме того, невозможно ссылаться (устанавливать агрегацию по ссылке) из другого объекта на объект, входящий в состав другого объекта. При проектировании классов запрещена (прямая или косвенная) рекурсивная агрегация объектов по значению с типом класса.

В языке Java вышеупомянутые проблемы исчезают сами собой. Но возникает следующий вопрос, весьма актуальный для обеспечения точного моделирования при проектировании [3]: как определить границу сущности в Java? Фактически ответ — никак. Все свойства с примитивным типом реализуются как поле такого типа, а все не примитивные — как указатели.

В Java отношение агрегации (моделирование частей-указателей) реализуется без проблем, поскольку это отношение основано на указателе. Проблема касается только отношения композиции, а точнее — определения физической границы моделируемого объекта. Далее рассматривается именно это отношение, то есть реализация части-значения рассматривается не посредством развертывания объекта-части в целое, а иным способом.

Фактически, если посмотреть на структуру памяти (сделать снимок) во время выполнения Java-программы, то, глядя на объект и на его часть, невозможно отличить, является ли это «часть-значение» или «часть-указатель», поскольку все части связаны с целым объектом через указатели.

Существует философская точка зрения [8], основанная на том, что отношение — первично, а сущность — вторично. Сущность в реальном мире может представлять собой некоторое отношение между бесконечным количеством компонентов этого отношения. При проектировании используются модели реального мира и абстрактные модели. С этой точки зрения, например, точка на плоскости представляет собой отношение двух действительных чисел, книгу можно смоделировать как некоторое отношение между названием, издательством, автором и содержанием, человека — как отношение между именем, датой рождения, адресом проживания и социальным идентификатором, автомобиль — как отношение между кузовом, двигателем, электрической системой, ходовой частью и

регистрационным свидетельством. На этом основана вся теория реляционных баз данных (реляционная модель данных) [7].

Из этого следует, что с философской точки зрения физическая граница определяет не целостность (неделимость составляющих частей) целого объекта, а тесную связь между этими составляющими частями, то есть тесное взаимодействие частей данного объекта как между собой, так и с окружающим целым объектом. Эти взаимодействия являются более сильными, чем взаимодействия между частями разных объектов. Интерфейс взаимодействия, поддерживающий эти отношения, является фиксированным. Таким образом, можно говорить о модульной структуре целого. Каждая часть — модуль — имеет интерфейс и сферу ответственности. Часть, входящую в состав целого объекта, можно заменить другой частью, если заменяющая часть поддерживает этот интерфейс.

15.2. РАСШИРЕННЫЕ ОБЪЕКТЫ

В языках с указателями существует понятие — развернутые объекты. Это понятие используется для моделирования сложных объектов, части которых «развернуты» за пределами физических границ. Развернутые объекты не в полной мере поддерживают модульность — а именно, динамическую взаимозаменяемость. Сложный объект, «созданный» с развернутыми внутренними частями, не может заменить внутреннюю часть на другой объект, реализующий тот же интерфейс, во время выполнения. В примере на C++ (рис. 15.2) можно сказать, что полиморфизм нарушен.

```
#include <iostream>
using std::cout; using
namespace std;

class A {
private:
    int x;
public:
    A() {}
    int getX() {return x;}
    void setX(int i) {x=i;}
};

класс B : public A {
private:
    int y;
public:
    B() { }
    B(int i) { y = i;}
```

```
    int getY() {return y;}
    void setY(int i) {y =
    i;}
};
class C {
    private:
        A a;
        int i; // какое-то поле
    public:
        C() { }
        A* getA() {return &a;};
        void setA(A a1) {a = a1;}
};
int main() {
    C c = C();
    B b1;
    b1.setX(10);
    b1.setY(12);
    cout << "Значения полей объекта перед полиморфным присваиванием:" << endl; cout
    << "Значение b1.x: " << b1.getX() << endl;
    cout << "Значение b1.y: " << b1.getY() << endl;
    // полиморфное присваивание
    c.setA(b1);
    // объявление указателя и преобразование типа из A* в B*
    A * a = c.getA();
    B * b2 = (B*)a;
    cout << "Значения полей объекта после полиморфного присваивания:" << endl; cout
    << "Значение b2.x: " << b2->getX() << endl;
    cout << "Значение b2.y: " << b2->getY() << endl; while
    (true) {};
    return 0;
}
```

Рис. 15.2. Пример 1

Источник: собственная работа

Программа из рис. 15.2 даёт следующий результат:

```
Значения полей объекта перед полиморфным присваиванием:
Значение b1.x:          10
Значение b1.y:          12
Значения полей объекта после полиморфного присваивания:
Значение b2.x:          10
Значение b2.y:          65535
```

Здесь объявляются три класса: класс **A**, его подкласс **B** и класс **C**, имеющий расширенное поле (с именем **a**). В теле функции **main** создается объект класса **C** (с именем **c**). При создании внутри этого объекта (класса **C**) будет развернут объект класса **A** (поле с именем **a**). Далее создаётся объект класса **B** (с именем **b1**), и в соответствующие поля **b1** будут присвоены значения **10** и **12**. Эти три значения будут выведены в консоль. Затем с помощью **c.setA(b1)** в

полю *a* (типа *A*) объекта с полиморфно присваивается объект класса *B* (с именем *b1*). С помощью выражения *A* a = c.getA()* создаётся указатель *a*, ссылающийся на поле *c.a*. Поскольку указатель *a* фактически ссылается на объект класса *B*, его можно привести к указателю, ссылающемуся на объект класса *B* (с именем *b2*). Значения полей этого объекта должны совпадать со значениями соответствующих полей *b1*, но, как видно из вывода программы, значение поля *y* отличается (не 12, а 65535).

Эта ситуация неудивительна, так как при создании объекта с память под этот объект выделяется с учётом расширенного объекта *a*. Затем, при полиморфном присваивании объекта *b1* объекту *a*, значение поля *y* не помещается в выделенное пространство для объекта *a*. Выводимое значение 65535 — результат обычной особенности (слабой типизации) языка C++.

Таким образом, при работе с расширенными объектами полиморфизм не работает, а модульность, повторноиспользуемость и объектно-ориентированное программирование в целом нуждаются в полиморфизме.

На самом деле современные агрегированные устройства созданы по принципам модульности. Например, автомобиль, хотя и содержит двигатель в пределах своих физических границ, может заменить этот двигатель на другой, например, на более современный, имеющий дополнительные компоненты двигателя. Важно, чтобы новый двигатель помещался в пространство старого и чтобы было возможно подключить все элементы интерфейса двигателя к остальным частям автомобиля. В случае с компьютером повсеместно применяется замена оперативной памяти, дискового хранилища или процессора. Сегодня то же самое можно сказать и об искусственных органах для человека. Приведенные выше примеры показывают, что физические границы не означают монолитность или целостность; скорее всего, они означают, что внутри границ устройства взаимосвязи между элементами более тесные, а внешние связи между элементами, находящимися внутри физических границ, и внешним миром слабы.

Физическая граница подчеркивает, что часть также является объектом и в принципе может рассматриваться как самостоятельная единица. Принадлежность некоторых компонентов отношений к охватываемому объекту статически нигде не указывается. Это определяется семантикой вышеупомянутых операций создания, присваивания, копирования (клонирования), сравнения и удаления.

Как видно из вышесказанного, для языков с указателями правильное моделирование физических границ (композиции) невозможно реализовать с помощью расширенных частей. Оказывается, что в Java всё продумано правильно, поскольку композиция реализуется на основе указателей.

Многие книги по языку Java [4, 5] обходят эту проблему проектирования стороной. В таких книгах, как [6], посвящённых проектированию, не раскрываются методы реализации композиции и агрегации. Мишель Яйзер [9] рассматривает композицию и простую агрегацию, основываясь исключительно на операции создания.

15.3. АГРЕГАЦИЯ

Агрегация и композиция должны реализовываться с помощью указателей. Когда реализуется часть-указатель (простая агрегация), она создаётся отдельно и передаётся по параметру (или каким-либо иным способом) в конструктор при создании целого — см. пример 2 на рис. 15.3.

```
class A { private
    int x; public
    A() {}
    public A(int x) {this.x = x;}
    public int getX() {return x;}
    public void setX(int x) {this.x = x;}
};

class B extends A {
    private int y;
    public B() { }
    public B(int y) {this.y = y;}
    public B(int x, int y) { super(x); this.y = y;} public
    int getY() {return y;}
    public void setY(int y) {this.y = y;}
}

class C {
    private A a;
    private int c; // какое-то поле
    public C(A a) {this.a = a;}
    public A getA() {return a;};
    public void setA(A a) {this.a = a;}

    public static void main(String[] args) {
        B b1 = new B(10, 12);
        C c = new C(b1);
        System.out.println("Значения полей объекта до изменения:"); System.out.println("Значение
        x: " + ((B)c.getA()).getX());
    }
}
```

```

        System.out.println("Значение y: " + ((B)c.getA()).getY());
        System.out.println("Значения полей объекта до изменения:"); b1.setX(24);
        b1.setY(48);
        System.out.println("Значения полей объекта после изменения:");
        System.out.println("Значение x: " + ((B)c.getA()).getX());
        System.out.println("Значение y: " + ((B)c.getA()).getY());
    }
}

```

Рис. 15.3. Пример 2

Источник: собственная работа

Программа из рис. 15.3 даёт следующий результат:

```

Значения полей объекта до изменения:
Значение b1.x: 10
Значение b1.y: 12
Значения полей объекта до изменения:
Значения полей объекта после изменения:
Значение b1.x: 24
Значение b1.y: 48

Значения полей объекта до изменения:
Значение b1.x: 10
Значение b1.y: 12
Значения полей объекта до изменения:
Значения полей объекта после изменения:
Значение b1.x: 24
Значение b1.y: 48

```

Здесь, если вы посмотрите на код класса **C**, то увидите, что при создании объекта класса **C** необходимо передать в конструктор объект класса **A**. Объект класса **C** со своим полем **a** включён в отношения агрегации (part-pointer). Как видно из основного кода, вместо объекта класса **A** мы можем полиморфно передать объект класса **B** (имеющий имя **b1**), поскольку класс **B** является подклассом класса **A**. Если мы изменим значения полей **b1**, то значения полей, к которым мы можем получить доступ через **c.getA()**, также изменятся.

15.4. Композиция

Как уже упоминалось выше, при композиции необходимо реализовать физическую границу для определения семантики операций создания, присваивания, копирования (клонирования), сравнения и удаления. Рассмотрим эти операции более подробно.

Построение (создание) композиции

С точки зрения композиции объект «рождается» со своей физической границей, то есть все его части создаются вместе с объектом, который их охватывает. Части целого не должны иметь других связей с внешней средой. Композитный объект можно создать, передав в конструктор в качестве параметра все его части. Пусть p_1 типа P_1 , p_2 типа P_2 , ..., p_n типа P_N — отдельные части объекта класса C , и кроме них объекты этого типа не имеют других частей. Тогда для конструктора класса C можно использовать шаблон из рис. 15.4.

```
class C { private
    P1 p1; private
    P2 p2;
    ... private
    PN pn;
    public C(P1 p1, P2 p2,..., PN pn) { // Конструктор, инициализирующий
        части: this.p1 = p1.clone();
        this.p2 = p2.clone();
    ...
        this.pN = pN.clone();
    }
    public C(..., P1 p1, P2 p2, ..., PN pn, ...) { // Другой конструктор C(p1, p2, ...,
        pn);
        ...
    }
}
```

Рис. 15.4. Пример 3

Источник: собственная работа

Все конструкторы должны вызывать конструктор класса, инициализирующий составные части. Чтобы предотвратить создание ссылок со стороны других объектов, не устанавливаются ссылки на переданные объекты-параметры, а вместо этого устанавливается ссылка на клоны этих объектов. Данный конструктор присваивает клоны объектов-составных частей полям объекта. Политика клонирования объектов p_1 , ..., p_n определяется соответствующими классами P_1 , ..., P_N .

Сравнение

Операции присваивания, сравнения и копирования (клонирования) являются бинарными операциями, в которых участвуют два объекта.

Результат операции сравнения имеет тип «логическое значение». Таким образом, эта операция является бинарным отношением эквивалентности.

В языках программирования для сравнения данных обычно используется оператор «==» (или «=» в языках типа Pascal). В языках с указателями этой операцией сравниваются все значения для частей-значений, а для частей-указателей сравниваются значения этих указателей. Как упоминалось ранее, таким образом определяются физические границы объекта. В С++ понятие строки (в её физических границах) включает в себя содержимое строки; поэтому, ввиду того, что строки в С++ реализованы в виде указателя, оператор «==» переопределён и сравнивает содержимое строки, а в языке С для этой цели используется функция `strcmp`. В языке Java эта операция используется для сравнения объектов примитивных типов. Для объектов использование этой операции имеет семантику сравнения адресов объектов, а не их значений. Для сравнения строк (объектов типа `String`) используются методы `equals`, `equalsIgnoreCase` и `compareTo`:

- `boolean equals(Object other)` используется для сравнения строк на точное совпадение их содержимого. Он возвращает `false`, если строки не совпадают. Этот метод переопределяет метод, унаследованный от класса `Object`. Подробнее об этом методе будет рассказано далее.
- `boolean equalsIgnoreCase(String other)` — текущая строка и строка, переданная в качестве параметра, сравниваются на совпадение без учёта регистра.
- `int compareTo(String other)` — текущая строка и строка, переданная в качестве параметра, сравниваются по лексикографическому порядку. Если строка меньше, возвращается `-1`; если больше — `1`; в противном случае (если они равны) — `0`.

Далее рассмотрим сравнение объектов в Java в более общем плане.

В книге [4] в разделе 5.2, посвящённом классу `Object`, речь шла о методе `equals`. Этот метод наследуется всеми классами, и когда возникает необходимость сравнить текущий объект, скажем, класса `A`, с каким-либо другим объектом, используется именно этот метод. Семантика сравнения (что будет сравниваться, с чем и как) раскрывается в этом методе, который переопределяет унаследованный метод. Тестовый паттерн, основанный на [4], представлен на рис. 15.5.

```
public boolean equals(Object o) {  
    // быстрая проверка на идентичность объектов if (this ==  
    o) return true;  
    // должен возвращать false, если явно переданный параметр  
    равен null if (o == null) return false;  
    // если классы не совпадают, они не могут быть равны if  
    (getClass() != o.getClass()) return false;  
    // теперь мы знаем, что otherObject — это не-null-  
    объект класса Employee A_other = (A)o;  
    // проверка, имеют ли поля одинаковые значения return //  
    проверка полей объекта;  
}
```

Рис. 15.5. Пример 4

Источник: [4]

Объект, который следует сравнить, пусть его назовем *o*, передаётся в метод в качестве параметра типа Object.

Обычно при переопределении этого метода для конкретного класса отличается только сравнение полей. Что с чем и как будет сравниваться, определяется именно здесь. Предположим, что некоторый (сравниваемый) объект класса *A* содержит в своих физических границах (в рамках отношения композиции) в качестве части какой-то другой объект класса *B*, на который, предположим, ссылается поле *f*. Тогда мы сравниваем эту часть с соответствующими частями сравниваемого объекта следующим образом:

```
...  
& getFieldF().equals(o.getFieldF())  
...
```

где `getFieldF()` возвращает указатель на объект с именем *f*.

Не допускается сравнение частей, не входящих в физические границы объекта. Обычно при сравнении двух компьютеров не обязательно сравнивать их владельцев, место покупки и т. п. При сравнении двух автомобилей, как правило, не требуется сравнивать их технические характеристики, адреса регистрации и т. п., конечно, если эти данные не входят в физические границы автомобиля. Сравнение должно быть реализовано на основе модели сущностей, в которой должны быть чётко определены физические границы объекта.

Кроме того, сравнение основано на свойствах отношений эквивалентности в математическом смысле:

- свойство рефлексивности: `x.equals(x) == true;`
- свойство симметрии: `x.equals(y) <=> y.equals(x);`
- свойство транзитивности: `x.equals(y) & y.equals(z) => x.equals(z);`

- свойство согласованности: разные вызовы `x.equals(y)` возвращают один и тот же результат, если между вызовами не произошло изменений со значениями величин, используемых в `equals`;
- свойство непустоты: `x.equals(null) == false`.

В [4] отмечается, что когда речь идёт об иерархии классов и необходимо сравнить объект класса с объектом его подкласса, чтобы не нарушить свойство симметрии, необходимо определить семантику сравнения объекта подкласса. Если семантика не изменяется, то есть эквивалентность объектов подкласса точно такая же, как и в суперклассе, то метод `equals` суперкласса должен использовать предикат `instanceof` и иметь квалификатор `final`. Если семантика изменяется, то подкласс переопределяет метод `equals`, а в суперклассе для проверки идентичности классов используется метод `getClass()`.

Вышеописанный метод сравнения можно назвать объектной эквивалентностью в том смысле, что объект сравнивает себя с другим объектом на основе семантики эквивалентности своего класса. Но можно говорить и об абстрактной (классовой) эквивалентности, когда два объекта сравниваются с какой-то абстрактной точки зрения. Например, можно сравнить карандаш и деревянную палку по длине и толщине, автомобиль и капсулу на колесе обозрения — по количеству мест и т. д. Для такого сравнения создаётся класс, соответствующий данной абстракции, и в этом классе реализуется метод `equals`. Этот метод сравнивает объекты с точки зрения семантики эквивалентности данного класса. Кроме того, если для сравнения требуются специальные методы доступа к значениям, создаётся интерфейс, объявляющий эти методы доступа для сравнения. Объекты любых классов, реализующих этот интерфейс, можно сравнивать с точки зрения данного класса.

Пример 5 на рис. 15.6 демонстрирует как объектное, так и абстрактное сравнение.

```
interface ExistX {
    int getX();
}

class EqualByX {
    public static boolean equals (ExistX o1, ExistX o2) {
        return o1.getX() == o2.getX();
    }
}
```

```
class SuperClass implements ExistX {
    private int x;
    public SuperClass() {}
    public SuperClass(int x) {this.x = x;}
    public int getX() {return x;}
    public void setX(int x) {this.x = x;} public
    boolean equals(Object otherObject) {
        // быстрая проверка на идентичность объектов if (this ==
        otherObject) return true;
        // должен возвращать false, если явно переданный параметр
        равен null if (otherObject == null) return false;
        // если классы не совпадают, они не могут быть равны
        if (!(otherObject instanceof SuperClass)) return false;
        // теперь мы знаем, что otherObject — объект класса SuperClass, не
        равный null SuperClass other = (SuperClass) otherObject;
        // проверка, имеют ли поля одинаковые значения return x ==
        other.x;
    }
    public static boolean equals (Object o1, Object o2) {
        if (!(o1 instanceof SuperClass) && (o2 instanceof SuperClass)) return false;
        SuperClass s1 = (SuperClass) o1;
        SuperClass s2 = (SuperClass) o2;
        return s1.getX() == s2.getX();
    }
};

class SubClass extends SuperClass { private
    int y;
    public SubClass() {}
    public SubClass(int y) {this.y = y;}
    public SubClass(int x, int y) { super(x); this.y = y;}
    public int getY() {return y;}
    public void setY(int y) {this.y = y;} public
    boolean equals(Object otherObject) {
        if (!super.equals(otherObject)) return false;
        // super.equals проверил, что this и otherObject принадлежат одному и тому же
        классу if (!(otherObject instanceof SubClass)) return false;
        // if (getClass() != otherObject.getClass()) return false;
        SubClass other = (SubClass) otherObject;
        return this.y == other.y;
    }
}

class Equals implements ExistX { private
    SuperClass a;
    private SubClass b;
    private int x;
    public Equals(int a, int b, int x) {
        this.a = new SuperClass(a);
        this.b = new SubClass(a, b);
        this.x = x;
    }

    public SuperClass getA(){return a;}
    public SubClass getB(){return b;} public
    int getX() {return x;}
    public static void main(String[] args) {
```

```

Equals e = new Equals(10, 12, 10);
System.out.println("---- Равенство а и b в объекте означает-----");
System.out.println("Объект а равен объекту b:" + e.getA().equals(e.getB()));
System.out.println("Объект b равен объекту а:" + e.getB().equals(e.getA()));
System.out.println("---- Равенство а и b на уровне объектов означает -----");
System.out.println("Объект а равен объекту b по методу EqualByX, что означает:" +
    EqualByX.equals(e.getA(), e.getB()));
System.out.println("Объект b равен объекту а в ExualByX, что означает:" +
    EqualByX.equals(e.getB(), e.getA()));
System.out.println("---- Равенство а и e в ExualByX означает -----");
System.out.println("Объект а равен объекту e в ExualByX означает: " +
    EqualByX.equals(e.getA(), e));
System.out.println("Объект e равен объекту а в ExualByX, что означает:" +
    EqualByX.equals(e, e.getA()));
System.out.println("---- Равность а и e в статическом методе equals ----
");
System.out.println("Объект а равен объекту e в ExualByX, что означает:" +
    SuperClass.equals(e.getA(), e.getB()));
System.out.println("Объект e равен объекту а по критериям ExualByX, то есть: " +
    SuperClass.equals(e.getB(), e.getA()));
}
}

```

Рис. 15.6. Пример 5

Источник: [4]

Программа из рис. 15.6 даёт следующий результат:

```

---- Равенство а и b в объектном смысле ----Объект а
равен объекту b:true
Объект b равен объекту а:false
---- Равенство а и b в объектном смысле ----Объект а равен
объекту b в смысле ExualByX: true Объект b равен объекту а в
смысле ExualByX: true
---- Равенство а и e по ExualByX означает ----Объект а равен
объекту e по ExualByX: true Объект e равен объекту а по
ExualByX: true
---- Равенство а и e в статическом методе equals ----Объект а
равен объекту e в ExualByX: true Объект e равен объекту а в
ExualByX: true

```

В коде на рис. 15.6 объявлен один интерфейс (**ExistX**) и объявлены классы **EqualByX**, **SuperClass**, **SubClass** и **Equals**. Рассмотрим процесс, реализованный в методе **main** класса **Equals**.

Создаётся объект **e** (класса **Equals**), в котором агрегированы объекты классов **SuperClass** и **SubClass** (с именами **a** и **b**). Кроме того, имеется поле **x**. Полученная схема объекта показана на рис. 15.7.

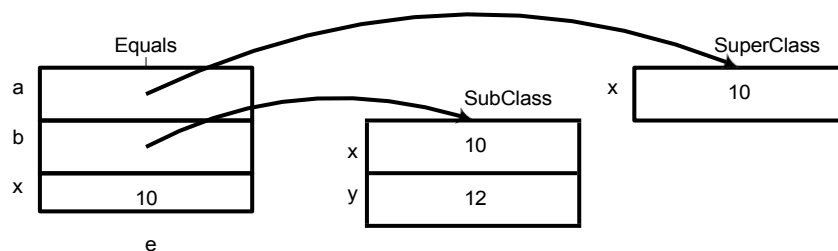


Рис. 15.7. Схема объектов, основанная на приведенной выше программе из рис. 15.6

Источник: собственная работа

С помощью выражения `e.getA().equals(e.getB())` объект, на который указывает поле `a`, сравнивается с объектом, на который указывает поле `b`. Сравнение объектов осуществляется на основе класса `SuperClass`. В этом классе переопределен метод `equals` класса `Object`. В данном случае для сравнения в качестве параметра передается объект его подкласса `SubClass`. При сравнении используется следующий алгоритм:

1. Если адреса объектов равны, то объекты эквивалентны
2. Если сравниваемый объект пуст, то объекты не эквивалентны
3. Если тип сравниваемого объекта не является полиморфным по отношению к `SuperClass`, то объекты не эквивалентны
4. В противном случае сравниваемый объект рассматривается как объект класса `SuperClass`, и сравниваются значения поля `x` (семантика сравнения класса `SuperClass`). Здесь не используется точное совпадение классов, чтобы обеспечить возможность полиморфного сравнения объектов подклассов.

Данное сравнение возвращает положительный результат после проверки четвертого элемента сравнения, то есть сравниваемые объекты эквивалентны. Далее, с помощью выражения `e.getB().equals(e.getA())` выполняется, казалось бы, симметричное сравнение — объект, на который указывает поле `b`, сравнивается с объектом, на который указывает поле `a`. Объекты сравниваются на основе класса `SubClass`, который переопределил метод сравнения в классе `SuperClass`, где сравнение будет осуществляться уже по следующему алгоритму:

1. Если они не эквивалентны по методу `equals` класса `SuperClass`, то объекты не эквивалентны
2. Если тип сравниваемого объекта не является полиморфным по отношению к `SubClass`, то объекты не эквивалентны. Здесь не используется точное совпадение классов, чтобы обеспечить полиморфное сравнение объектов подкласса.
3. В противном случае сравниваемый объект рассматривается как `SubClass`, и сравниваются значения полей поля `y` (семантика сравнения класса `SubClass`).

Исходя из значений, указанных в программе, объекты не эквивалентны, поскольку объект класса `SuperClass`, переданный в качестве параметра, не имеет поля с именем `y`. Следует напомнить, что эти два отношения не тождественны: первое — это сопоставимость двух объектов на основе класса `SuperClass`, второе — сопоставимость двух объектов на основе класса `SubClass`, поэтому здесь не может быть симметрии: `SuperClassEquality(a, b) ≠ SubClassEquality(A, b)`.

С помощью `EqualByX.equals(e.getA(), e.getB())` объекты, обозначенные полями `a` и `b` соответственно, сравниваются абстрактно с точки зрения класса `EqualByX`. Этот класс может сравнивать объекты типа `ExistX`. Поскольку и класс `SuperClass`, и класс `SubClass` реализуют этот интерфейс, их объекты можно сравнивать с использованием семантики сравнения данного класса. Оба объекта имеют поле `x`, и значение этого поля одинаково, поэтому с этой точки зрения они эквивалентны. Следующее сравнение — `EqualByX.equals(e.getB(), e.getA())` — является симметричным по отношению к вышеуказанному сравнению; оба сравнения выполняются на основе класса `EqualByX` и, следовательно, дадут одинаковый результат.

Следующие два сравнения также являются абстрактными, основанными на классе `EqualByX`, но здесь весь объект (`e`) сравнивается со своей частью (`a`) с использованием семантики сравнения класса `ExistX`. С этой точки зрения оба сравнения (прямое и симметричное) дадут утвердительный результат.

В последних сравнениях объекты, обозначенные полями **a** и **b**, абстрактно сравниваются с точки зрения класса **SuperClass**. Этот класс, помимо стандартного метода сравнения **equals**, унаследованного от **Object** и переопределённого в нём самом, имеет статический метод сравнения **equals**, которому передаются два объекта типа **Object**. Алгоритм сравнения следующий:

1. Если типы сравниваемых объектов не являются полиморфными по отношению к **SuperClass**, то объекты не эквивалентны
2. В противном случае объекты рассматриваются как **SuperClass**, и сравниваются значения поля **x** (семантика сравнения класса **SuperClass**). Здесь не используется проверка точного совпадения классов, чтобы обеспечить возможность полиморфного сравнения объектов подклассов.

Эти сравнения (прямое и симметричное) дадут положительный результат.

В стандартной библиотеке Java — **java.util.Arrays** — реализован статический метод **static boolean equals(type [] a, type [] b)** для сравнения массивов примитивных типов, то есть типов **int**, **long**, **short**, **char**, **byte**, **boolean**, **float**, **double**.

Присваивание и копирование (клонирование)

Присваивание и копирование объектов — это схожие операции. В C++ существует присваивание по умолчанию, при котором копируются все развернутые части объекта, но вы можете переопределить эту операцию, придав ей иную семантику. Семантика присваивания объектов и передачи параметров по значению идентична. Операция копирования по умолчанию (конструктор копирования по умолчанию) и присваивание имеют одинаковое значение. Вы также можете переопределить конструктор копирования по умолчанию.

В Java при присваивании копируется только адрес объекта. В следующем фрагменте:

```
A original = new A();  
A copy = original;
```

переменные **original** и **copy** ссылаются на один и тот же объект — рис. 15.8.

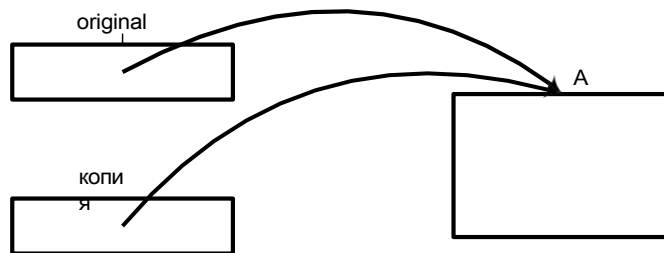


Рис. 15.8. Указатели переменных при простом копировании

Источник: собственная работа

Для копирования частей, входящих в физические границы (клонирования), необходимо использовать метод `clone()` класса `Object` с доступом `protected` или прибегнуть к сериализации (линеаризации). Рассмотрим эти подходы по отдельности.

Согласно стандартной документации JDK 8 [10], метод `clone` имеет следующую сигнатуру:

```
protected Object clone() throws CloneNotSupportedException
```

Для клонирования объекта определённого класса необходимо, чтобы этот класс реализовывал интерфейс `Cloneable`. Этот интерфейс не обязывает класс реализовывать какие-либо методы, необходимо лишь проверить, что формальная реализация данного интерфейса в указанном классе (эта проверка выполняется в методе `clone()`) прошла успешно. Вызов `x.clone()` создаёт новую копию (клон) указанного объекта, на который указывает переменная `x`, но точное значение этого клонирования зависит от класса объекта, на который указывает `x`. Согласно приведённой выше документации, при переопределении этого метода возвращаемый клон должен быть получен с помощью `super.clone()`.

Затем выполняется следующее выражение:

- `x.clone() != x.` // Это обеспечивается классом `Object`.
- `x.clone().getClass() == x.getClass()`
- `x.clone().equals(x)`

В документации [10] и книгах по Java, например [4], говорится о глубоком и поверхностном клонировании. Фактически в процессе создания необходимо клонировать объект в пределах его физических границ.

Чтобы обеспечить независимость клонированного объекта от исходного в пределах физических границ объекта, в случае необходимости может потребоваться выполнить некоторое «изменение» полученного объекта после **вызова** `super.clone()`. Если объект содержит подобъект, не входящий в его физические границы, то ничего делать не нужно; в противном случае, то есть если подобъект попадает в его физические границы, необходимо скопировать эту часть и перенаправить указатель клонированного объекта (рис. 15.9).

На рис. 15.9 в исходном объекте «original» прямоугольники зелёного цвета обозначают части этого объекта, входящие в его физические границы. Эти части обозначены пунктирной зелёной линией. Жёлтый цвет обозначает объекты, не входящие в его физические границы. Результат клонирования обозначен как «clone»).

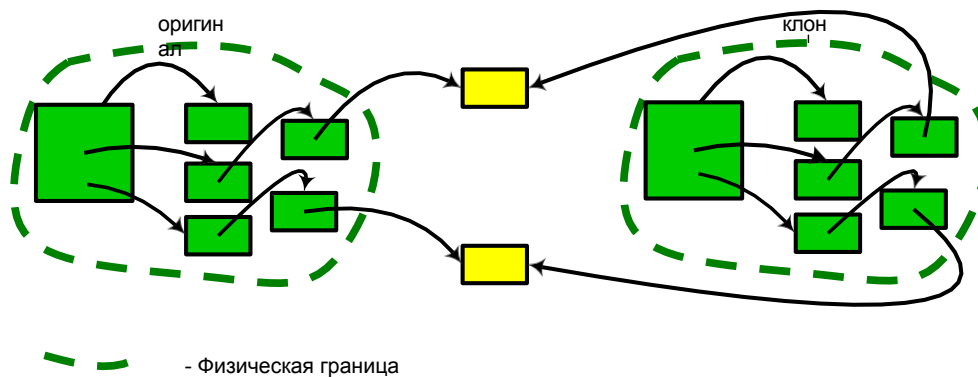


Рис. 15.9. Клонирование сложного объекта

Источник: собственная работа

В примере 6 (рис. 15.10) показано клонирование в пределах физической границы.

```
пакет Examples;  
public class ForClone implements Cloneable{  
    private int intField;  
    public ForClone (int i) {  
        intField = i;  
    }  
}
```

```

    }
    public ForClone clone() throws CloneNotSupportedException { ForClone
        cloned = (ForClone)super.clone();
        return cloned;
    }
    public void setInt (int i) {intField = i;}
    public int getInt() {return intField;}
}

пакет Examples;
public class Cloning implements Cloneable{ private
    double doubleField;
    private ForClone forField;
    public Cloning (int i, double d) {
        forField = new ForClone(i); doubleField
        = d;
    }
    public Cloning clone() throws CloneNotSupportedException {
        Cloning cloned = (Cloning)super.clone(); cloned.forField =
        forField.clone();
        return cloned;
    }
    public static void main (String[] args) throws CloneNotSupportedException {
        Cloning sc = new Cloning(12, 12.12);
        Cloning scNew = (Cloning)sc.clone();
        scNew.doubleField = 24.24;
        scNew.forField.setInt(24);
        System.out.println("Оригинал = " + sc.doubleField);
        System.out.println("Клон = " + scNew.doubleField);
        System.out.println("Оригинал = " + sc.forField.getInt());
        System.out.println("Клон = " + scNew.forField.getInt());
    }
}

```

Рис. 15.10. Пример 6

Источник: [4]

Создаётся метод `main (sc)` и его клон (`scNew`). После клонирования поля клона корректируются: поле простого типа `double (doubleField)` и поле типа `ForClone (forField)`. Поскольку поле `forField` входит в физические границы данного объекта, оно также копируется. Для этого вызывается метод `clone` данного объекта, который знает, как выполнять клонирование.

Как видно из результата, корректировка затрагивает только клон, а значения полей исходного объекта не изменяются:

```

Original = 12.12
Клон = 24,24
Оригинал = 12
Клон = 24

```

При работе с массивами функцию `clone()` также можно применять к массивам. В этом случае значения элементов массива копируются в вновь созданный массив. Таким образом, если элементы массива имеют примитивный тип, то копируется это примитивное значение; если же — объект (то есть указатель), то копируется указатель, который тем самым указывает на один и тот же объект как в исходных, так и в клонированных элементах. В качестве примера можно привести фрагмент с рис.

15.11: продолжение метода `main`.

```
Cloning[] scArr = new Cloning[4];
scArr[0] = sc;
scArr[1] =
sc;   scArr[2]
=     sc;
scArr[3] =
sc;
for (Cloning c : scArr) {
    System.out.println("Исходный: double = " + c.doubleField);
    System.out.println("Исходный:   forField.int = "+c.forField.getInt());
}
Cloning[] scArrClone = scArr.clone();
scArrClone[1].doubleField = 36.36;
scArrClone[1].forField.setInt(36); for
(Cloning c : scArr) {
    System.out.println("Исходные данные: double = " + c.doubleField);
    System.out.println("Исходные данные: forField.int = " + c.forField.getInt());
}
for (Cloning c : scArrClone) { System.out.println("Клон:
double = " + c.doubleField);
    System.out.println("Клон: forField.int = " + c.forField.getInt());
}
```

Рис. 15.11. Пример 7

Источник: [4]

Здесь после клонирования массива значения первого элемента клонированного массива корректируются. После этого выделяются все элементы исходного и клонированного массивов. Как видно из результата, исходный массив также изменяется:

```
Источник double = 12.12
:
Источник   forField.int = 12
:
Источник double = 12,12
:
Источник   forField.int = 12
:
Источник double = 12,12
:
Источник   forField.int = 12
:
Источник double = 12,12
:
Источник   forField.int = 12
:
Источник double = 12,12
:
Источник   forField.int = 12
:
Исходный текст: double = 36.36
Исходный текст: forField.int =
36 Исходный текст: double =
36.36 Исходный текст:
forField.int = 36 Исходный
текст: double = 36.36 Исходный
текст: forField.int = 36
```

```

Исходное: double = 36,36
Исходное: forField.int = 36
Клон: double = 36.36 Клон:
forField.int = 36 Клон:
double = 36.36 Клон:
forField.int = 36 Клон:
double = 36.36 Клон:
forField.int = 36 Клон:
double = 36.36 Клон:
forField.int = 36

```

При клонировании массивов вся ответственность за «соблюдение» физических границ лежит на классе, содержащем массив внутри себя.

Другой подход к клонированию заключается в линеаризации. При линеаризации объект сначала преобразуется в массив байтов, а затем из него заново создаётся объект. Полученный объект будет клоном исходного объекта. Класс, объект которого копируется (включающий объект), а также все классы, объекты которых входят в состав включающего объекта, должны реализовывать интерфейс `Serializable`. Этот интерфейс не обязывает класс реализовывать какие-либо методы, как это делает интерфейс `Cloneable`, но единственное отличие заключается в том, что при клонировании необходимо переопределить метод `clone()`, а при линеаризации ничего переопределять не нужно. Пример 7 (рис. 5.11) реализован с помощью линеаризации по коду из рис. 5.12.

```

package Examples;
import java.io.Serializable;

public class ForClone implements Serializable { private
    int intField;
    public ForClone (int i) {
        intField = i;
    }
    public void setInt (int i) {intField = i;}
    public int getInt() {return intField;}
}

пакет Examples; импорт
java.io.*; импорт
java.util.*;

public class Serializing implements Serializable{
    private double doubleField;
    private ForClone forField;
    public Serializing (int i, double d) {
        forField = new ForClone(i);
        doubleField = d;
    }
    // Инструмент клонирования

```

```
public Object clone() { try
{
    // сохранение объекта в массив байтов
    ByteArrayOutputStream bout = new ByteArrayOutputStream();
    ObjectOutputStream out = new ObjectOutputStream(bout);
    out.writeObject(this);
    out.close();
    // чтение клона объекта из массива байтов
    ByteArrayInputStream bin = new ByteArrayInputStream(bout.toByteArray());
    ObjectInputStream in = new ObjectInputStream(bin);
    Object ret = in.readObject();
    in.close();
    return ret;
} catch (Exception e) {
    return null;
}
}

public static void main (String[] args) throws CloneNotSupportedException { Serializing
    sr = new Serializing(12, 12.12);
    Serializing srNew = (Serializing)sr.clone();
    srNew.doubleField = 24.24;
    srNew.forField.setInt(24);
    System.out.println("Исходное значение doubleField = " + sr.doubleField);
    System.out.println("Значение doubleField в клоне = " + srNew.doubleField);
    System.out.println("Исходные значения forField.intField = " + sr.forField.getInt());
    System.out.println("Значения forField.intField в клоне = " + srNew.forField.getInt());
}
}
```

Рис. 15.12. Пример 7 с изменениями

Источник: [4]

Результат такой же, как и в предыдущем примере:

```
Исходное значение doubleField =
12.12 Клон doubleField = 24.24
Исходное значение forField.intField
= 12 Клон forField.intField = 24
```

15.5. ЗАКЛЮЧЕНИЕ

В Java удаление составных объектов очень просто благодаря автоматической сборке мусора. Когда ни один из указателей не указывает на объект, он автоматически удаляется. Можно сказать, что для Java-программиста такая задача вообще не возникает.

СПИСОК ЛИТЕРАТУРЫ

- [1] Буч Г., Максимчук Р., Энгл М., Янг Б., Коналлен Дж. и Хьюстон К. (2007) *«Объектно-ориентированный анализ и проектирование с приложениями»*. 3-е изд. Addison-Wesley, 720 с.
- [2] Фаулер М. (2000) *Краткое руководство по стандартному языку объектного моделирования (3-е изд.)*. Addison-Wesley, 181 с.
- [3] Мейер Б. (1997) *Объектно-ориентированное построение программного обеспечения*. 2-е изд., Prentice Hall, 770 с.
- [4] Хорстманн К. и Корнелл Г. (2012) *«Core Java»*. 9-е изд., Prentice Hall, т. 1, 2.
- [5] Дейтель П. и Дейтель Х. (2009) *«Java™ для программистов: серия Deitel® Developer»*. Prentice Hall, 1200 с.
- [6] Гилберт С. и Маккарти Б. (1998) *Объектно-ориентированное проектирование на Java*. SAMS, 731 с.
- [7] Коннолли Т. и Бегг К. (2002) *Системы баз данных. Практический подход к проектированию, реализации и управлению*. 3-е изд. Addison-Wesley, 1440 с.
- [8] Чанышев А. Н. (1981) *Курс лекций по древней философии: Учебное пособие для философских факультетов и отделений университетов*. Москва, Высшая школа (на русском языке).
- [9] <http://www.adobe.com/devnet/actionscript/learning/ooop-concepts/composition-and-aggregation.html>
- [10] *Документация JDK*. <http://docs.oracle.com/javase/specs/jls/se8/jls8.pdf>